Шаг 06. Первый рабочий прототип: CLI

Блок 1 · Предыдущий: 05 · Следующий: 07. База данных

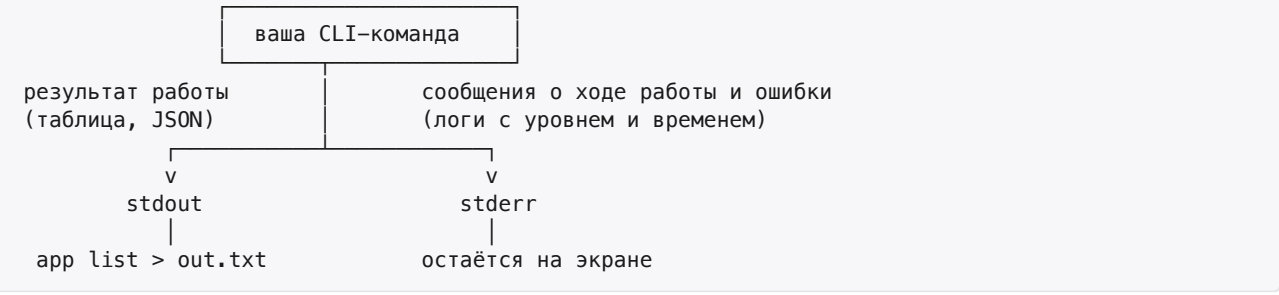
> **Время:** 8–12 часов (7–10 занятий). Обоснование — TIME.md.
> Незнакомые слова — словарь.
> Это последний шаг блока 1: после него у вас появится приложение, которое можно показать.

Зачем этот шаг

К концу шага у вас появится первое приложение, которое можно запустить и показать. Веб-интерфейс здесь не нужен: командная строка даёт рабочий прототип без лишних сущностей и остаётся в проекте навсегда как инструмент администрирования.

Теория

- **CLI** — интерфейс командной строки. В реальных проектах через него делают миграции, импорт данных, разовые операции обслуживания.
- **Typer** — библиотека для создания CLI из обычных функций: аргументы и их типы берутся из аннотаций, справка генерируется автоматически.
- **Rich** — форматированный вывод в терминал: таблицы, цвета, прогресс-бары.
- **logging** — стандартный модуль журналирования. В отличие от `print`, умеет уровни важности, разные приёмники и настраивается без правки кода.
- **stdout и stderr** — два отдельных потока вывода. В первый идёт результат работы, во второй — сообщения о ходе работы. Благодаря этому результат можно перенаправить в файл, не смешав с логами.



Проверить это просто: `app list > out.txt` должен положить в файл только результат, а логи оставить в терминале. Если в файл попали логи — вывод перемешан.

- **Коды возврата** — успешное завершение это 0, ошибка — ненулевой код. По нему CI и скрипты понимают, что произошло.

Typer: команды, аргументы и опции

```
from typing import Annotated
import typer

app = typer.Typer(help="Трекер поиска работы.")

@app.command("list")
def list_applications(
    status: Annotated[str | None, typer.Option(help="Фильтр по статусу")] = None,
    limit: Annotated[int, typer.Option("--limit", "-n", min=1)] = 20,
) -> None:
    """Показать сохранённые отклики."""
    ...

@app.command()
def move(application_id: int, status: str) -> None:
    """Перевести отклик в новый статус."""
    try:
        services.move(application_id, status)
    except TransitionError as err:
        err_console.print(f"Ошибка: {err}")
        raise typer.Exit(code=1)
```

Параметр без значения по умолчанию становится **аргументом**: позиционным и обязательным (`app move 7 applied`). Параметр с `typer.Option` или значением по умолчанию — **опцией** с именем (`--limit 5`). Типы берутся из аннотаций: `--limit abc` Typer отклонит сам с кодом 2 и понятным сообщением. Докстринг функции становится текстом `--help`. Команду в терминале создаёт строка в `[project.scripts]`, указывающая на объект `app`.

Rich: вывод результата

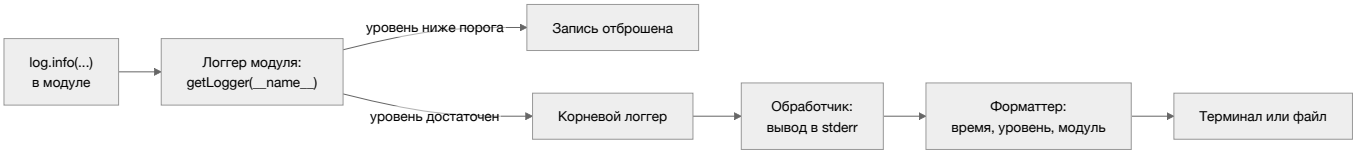
```
from rich.console import Console
from rich.table import Table

console = Console()                    # stdout: результат работы
err_console = Console(stderr=True)     # stderr: ошибки и подсказки

table = Table("ID", "Вакансия", "Статус")
for item in items:
    table.add_row(str(item.id), item.title, item.status)
console.print(table)
```

Rich сам определяет, что вывод перенаправлен в файл, и не пишет туда цветовые коды.

Как устроен logging



Логгеры образуют дерево по именам модулей, записи поднимаются к корневому логгеру, а куда и в каком виде их выводить, решают обработчики. Отсюда разделение обязанностей: модуль библиотеки только создаёт логгер и пишет в него, а настраивает вывод один раз точка входа — CLI или API.

```
log = logging.getLogger(__name__)          # в каждом модуле

def load(path: Path) -> list[Record]:
    log.info("чтение %s", path)            # аргументы, а не f-строка
    ...

def configure_logging() -> None:           # только в точке входа
    logging.basicConfig(
        level=os.environ.get("LOG_LEVEL", "INFO"),
        format="%(asctime)s %(levelname)s %(name)s: %(message)s",
        stream=sys.stderr,
    )
```

Аргументы вместо f-строки откладывают форматирование: если уровень отключён, строка не собирается. `log.exception("...")` внутри `except` добавляет в запись traceback.

Уровень	Когда
`DEBUG`	подробности для разработчика: параметры, промежуточные значения
`INFO`	нормальные события: команда выполнена, загружено 50 записей
`WARNING`	что-то необычное, но работа продолжается: источник ответил медленно
`ERROR`	операция не выполнена
`CRITICAL`	приложение не может продолжать работу

Коды возврата и тесты CLI

Код 0 — успех, 1 — общая ошибка, 2 по соглашению Click и Typer — неверные аргументы. Последний код видно командой `echo $?` (в PowerShell — `$LASTEXITCODE`), а оболочка выполняет правую часть `app load && app list` только после успеха левой.

```
from typer.testing import CliRunner

runner = CliRunner()

def test_list_respects_limit():
    result = runner.invoke(app, ["list", "--limit", "2"])
    assert result.exit_code == 0
    assert "Python-разработчик" in result.stdout
```

argparse, Click и Typer

	argparse	Click	Typer
Где	стандартная библиотека	отдельный пакет	надстройка над Click
Как описывают	вызовы `add_argument`	декораторы	аннотации типов функции
Сильная сторона	нет зависимостей	зрелость, расширения	меньше кода, проверка типов

Материалы

- Typer: Tutorial — команды, аргументы, опции.
- Rich: документация — разделы Console и Tables.
- Logging HOWTO — уровни и настройка.
- Logging Cookbook — как справочник по нетипичным случаям.
- Typer: Testing — `CliRunner`, вывод и код возврата.

- argparse — обзор встроенной альтернативы; писать вторую CLI не нужно.

Задание

1. Соберите CLI для первого сценария из паспорта: загрузить фиксированные данные, показать список, выполнить доменную операцию (например, сменить статус) и увидеть результат.
2. Данные берутся из источника шага 04.1. Для нескольких отдельных запусков сохраняйте их в JSON; вариант в памяти допустим для команды `demo`, показывающей весь сценарий за один запуск.
3. Список выводите таблицей через Rich, добавьте опции фильтрации и ограничения количества.
4. Настройте логирование: уровень задаётся флагом или переменной окружения, логи идут в stderr, сообщения содержат время, уровень и контекст.
5. Обработайте ошибки: пользователь видит понятное сообщение, а не traceback, процесс завершается ненулевым кодом.
6. Напишите тесты на команды CLI, проверяя код возврата и вывод.
7. Опишите в README все команды с примерами запуска.
8. **По желанию:** объясните, какие задачи решают argparse, Click и Typer, и обоснуйте выбор.

Обязательные критерии

- ☐ `--help` для приложения и для каждой команды содержит внятные описания.
- ☐ Все команды работают на данных из заглушки, приложение запускается одной командой.
- ☐ В коде библиотеки нет ни одного `print` — только логирование и вывод в слое CLI.
- ☐ Уровень логирования переключается снаружи, без правки кода.
- ☐ Ошибка источника даёт понятное сообщение и ненулевой код возврата.
- ☐ Команды покрыты тестами, README содержит примеры запуска.
- ☐ Основной локальный сценарий паспорта показан одной воспроизводимой последовательностью команд.

Желательные критерии

- ☐ Логи идут в stderr, результат работы — в stdout; проверено перенаправлением в файл.
- ☐ Объяснён выбор между argparse, Click и Typer.
- ☐ В выводе списка есть и фильтр, и ограничение количества.

На что обратить внимание

- Логика остаётся в доменных модулях: CLI только разбирает аргументы и печатает результат.
- Не логируйте пароли, токены и персональные данные — привычку лучше закладывать сразу.
- Это первый шаг, результат которого можно показать другому человеку. Покажите: обратная связь на прототипе полезнее ещё одного рефакторинга.

Вопросы для собеседования

Проверьте себя по темам этого шага. Ответов в программе нет намеренно: формулировка своими словами и есть проверка. Где можно — отвечайте на примере своего кода.

1. Зачем backend-проекту командный интерфейс, если есть API?
2. Чем `logging` лучше `print`? Назовите три различия.
3. Какие бывают уровни логирования и как выбрать подходящий?
4. Почему логи идут в stderr, а результат работы — в stdout?
5. Что означает код возврата процесса и кто его читает?
6. Что должен увидеть пользователь при ошибке: traceback или сообщение? Почему?
7. Какие задачи решают argparse, Click и Typer? Чем обоснован ваш выбор?
8. Как тестировать команду CLI: что именно проверяется?
9. Почему в библиотечном коде не должно быть `print`?
10. Что нельзя писать в логи и почему привычку закладывают сразу?
11. Почему логика остаётся в доменных модулях, а не в коде команд?
12. Как задать уровень логирования снаружи, не правя код?

Полный набор по блоку — вопросы по блоку 1.